

Master Claude Code

The Complete Guide & Cheat Sheet for Everyone

1: Why Claude Code Changes Everything

Chat window

Upload limits

Sessions expire

You do everything

Claude Code

Full system access

All files, any size

Hours-long tasks

AI teammates

From Assistant to Operating System — AI that runs across your entire workflow

3: The Claude Code Workflow



7: Skills & CLAUDE.md

Skills (Reusable Automations):

Task-specific instructions Claude auto-loads.

```
.claude/skills/  
├── review-code/  
│   └── SKILL.md  
├── deploy/  
│   └── SKILL.md  
└── generate-tests/  
    └── SKILL.md
```

Rules (.claude/rules/name.md):

```
---  
description: What this enforces  
globs: ["src/**/*.ts"]  
---  
  
- Named exports only  
- No console.log in production
```

CLAUDE.md (Project Memory):

```
# Marketing Dashboard  
## Commands  
- npm run build  
- npm test  
## Conventions  
- TypeScript strict mode  
- Follow existing patterns  
## Important  
- Main data: /data/analytics.csv  
- Never modify legacy/ folder
```

Location: Global: ~/ .claude/CLAUDE.md • Project: ./CLAUDE.md

Hooks (auto-run on events):

```
{  
  "hooks": {  
    "PostToolUse": [{  
      "matcher": "Edit|Write",  
      "hooks": [{  
        "type": "command",  
        "command": "npm run lint --quiet"  
      }]  
    }]  
  }  
}
```

2: What Claude Code Can Do

Full File System Access
Read, write, create, organize any file
"10,000 files in minutes"

Tool & Command Execution
Run commands, scripts, Git, packages
"Non-coders running bash"

MCP Connections (200+)
GitHub, Slack, databases, APIs
"One interface for everything"

Multi-Agent Work
Parallel subagents, background tasks
"Like having 5 employees"

4: Getting Started

Install:

macOS / Linux
curl -fsSL https://claude.ai/install.sh | bash
Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex

Authenticate:

Subscription (opens browser)
claude
API key
export ANTHROPIC_API_KEY="sk-ant-..."

Requirements:

• macOS 13+, Ubuntu 20.04+, Windows 10+
• Pro (\$20/mo), Max (\$100/mo), or API
• Windows: Git for Windows required

First Session:

mkdir my-project
cd my-project
claude

Verify:

claude --version
claude doctor

8: Prompting Techniques

TECHNIQUE	WHEN TO USE	EXAMPLE
Be Specific	First ask	"Create REST API with JWT auth and PostgreSQL"
Give Context	Complex tasks	"This is a React 18 app using TypeScript..."
Match Patterns	Adding features	"Follow the pattern in auth.ts"
Constrain	Quality control	"Keep behavior identical. Run tests after."
Iterate	Building up	"Now add error handling to that"
Persist	Standards	"Add to CLAUDE.md: use named exports"

The Prompt Formula:

Contextwhat to look at — "read this file", "look at git diff"

Taskwhat to do — "fix it", "add tests", "refactor"

Constraintshow — "follow patterns", "run tests", "keep behavior"

5: Connect Everything (MCP)

Model Context Protocol — USB-C for AI

GitHub

Sentry

Linear

GitLab

Slack

Notion

PostgreSQL

Playwright

Context7

Discord

claude mcp add <name> -- npx -y @scope/server

List: claude mcp list • Remove: claude mcp remove <name>

6: Essential Commands

COMMAND	WHAT IT DOES	SHORTCUT	ACTION
/help	All commands	Ctrl+C	Cancel
/status	Auth & usage	Ctrl+D	Exit
/model	Switch model tier	Esc x2	Rewind
/compact	Compress to save tokens	Up/Down	History
/clear	Reset context		
/mcp	MCP server status	@file	@report.csv
/doctor	Diagnose issues	@folder/	@data/
/logout	Log out		

9: Model Routing & Pro Tips

TASK TYPE	TIER	WHY
Quick search, scanning	Fast	Fast, cheap
Daily coding, features	Balanced	Balanced
Architecture, complex bugs	Most-capable	Most powerful

Switch: /model or claude --model

Pro Tips
→ Paste the FULL error with stack trace
→ Say "follow existing patterns" for consistency
→ Add "run tests after" to catch regressions
→ Create CLAUDE.md on day 1 of every project
→ Use /compact when context gets long
→ Use @filename to reference files directly
→ "Write tests FIRST" before refactoring

1 The "Onboard Me"

Explore this codebase and give me:

1. What it does (one paragraph)
2. Tech stack
3. Entry point
4. How to run it

→ Oriented in 30 seconds, not 30 minutes

2 The "Do It Like the Rest"

Add [feature]. Look at how [existing feature] is implemented and follow the exact same patterns – structure, naming, error handling, test style.

→ Consistency > cleverness

3 The "Fix This Error"

I'm getting this error:
[paste FULL error + stack trace]

Read the relevant files, explain what's causing it, fix it, and run the tests.

→ Full stack trace gives Claude file paths + line numbers

4 The "Review My Changes"

Review git diff for:

- Bugs or logic errors
- Security issues
- Missing error handling
- Things I might have forgotten

Be specific about line numbers.

→ Catches what your eyes miss after hours of coding

5 The "Write Tests"

Write tests for [file] covering:

- Happy path (normal usage)
- Edge cases (empty, null, boundaries)
- Error cases (what should fail)

Follow the existing test patterns.

→ 3 categories = comprehensive, not just happy-path

6 The "Explain This"

Explain [function/file/module]. Walk through step by step as if I know the language but have never seen this codebase before.

→ "Know the language" prevents over-explaining syntax

7 The "Production Ready"

Review for production readiness:

- Error handling
- Security issues
- Input validation
- Rate limiting
- Logging
- Env vars documented

Fix critical issues. List the rest.

→ Checklist in the prompt = nothing gets skipped

8 The "Safe Refactor"

Refactor [file] for readability.
Rules:

- Keep behavior identical
- Run tests after each change
- If no tests, write them FIRST

→ "Tests FIRST" prevents refactoring untested code

9 The "Full Commit"

Look at all changes (git diff). Create a commit with a clear message explaining what changed and why.

→ Claude writes better commit messages — it sees the full diff

10 The "Set Up My Project"

Create a CLAUDE.md based on:

- Actual tech stack and dependencies
- Existing coding patterns
- Test setup and build commands

Then create .claude/rules/ for conventions.

→ Auto-generated from real code beats writing from scratch

🔥 BONUS: THE META-PROMPT

I want to [goal]. I'm not sure the best approach. Can you:

1. Look at the codebase
2. Suggest 2-3 approaches
3. Recommend the best one and explain why
4. Then implement it

Use this when you don't know how to ask for something.

📄 QUICK CLAUDE.MD TEMPLATE

```
# Project Name

## Stack
- [Language], [Framework], [Database]

## Commands
- Build: `npm run build`
- Test: `npm test`
- Dev: `npm run dev`

## Conventions
- [Your coding preferences]
- [Naming rules]

## Important
- [Things Claude should always/never do]
```

10: Resources & Pricing

FREE COURSE \$0 15 modules • 9+ hours	BUNDLE \$59.99 8 modules + 11 projects
PROJECTS \$39.99 11 builds	ADVANCED \$29.99 8 modules

- Links:
- [Free Course on GitHub](#)
 - [Complete Bundle](#)
 - [Official Claude Code Docs](#)